CSE 4345: Big Data Analytics

Week 6, Part 1 — Apache Spark: RDDs, DataFrames & In-Memory Computing

Department of Computer Science & Engineering
Premier University, Chittagong

Semester 2026

Course & Lecture Information

Lecture Identity

Course:	CSE 4345
Week / Part:	6 of 11 | Part 1 of 2
CLO:	CLO3
PLO:	PLO(e), PLO(f)

CLO3

"Explain big data techniques, tools, and technologies used to store, manage, and analyse large datasets"

Course Progression

Week 4: MapReduce, Pig, Hive, HBase
Week 5: YARN, file formats, compression

Part 1 Covers

- Why MapReduce fails for iterative algorithms
- Spark history and the in-memory insight
- Spark architecture: Driver, Executors, Cluster Manager
- RDD: the core abstraction (immutable, partitioned, lineage)
- RDD lineage and fault-tolerant recomputation
- Transformations vs. actions; lazy evaluation
- Narrow vs. wide dependencies; stages and tasks
- PySpark programming examples
- Spark SQL and the DataFrame API
- Spark ecosystem (MLlib, Structured Streaming)

Lecture Agenda

1. The MapReduce Bottleneck
2. Apache Spark: Architecture & RDDs
3. Transformations, Actions, and Lazy Evaluation
4. PySpark Programming
5. Spark SQL and DataFrames
6. The Spark Ecosystem
7. MapReduce vs. Spark: Head-to-Head
8. Ivy League Alignment
9. Student Assessment Pool

The MapReduce Bottleneck

Why MapReduce Fails for Iterative Algorithms

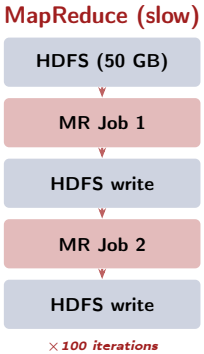
The Fundamental Problem

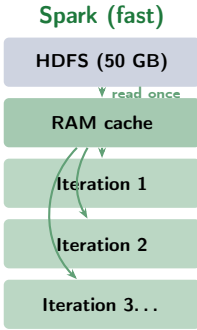
MapReduce writes **every intermediate result to HDFS** before the next step can begin. For algorithms that must pass over the same data repeatedly (machine learning, graph algorithms), this incurs enormous disk I/O cost.

K-Means Clustering: 100 Iterations

Each iteration: read training data → compute new centroids → write to HDFS.

- Dataset: 50 GB on HDFS
- **MapReduce**: 100 iterations × (50 GB read + 50 GB write) = **10 TB disk I/O**
- **Spark**: iteration 1 reads 50 GB from HDFS and caches in RAM. Iterations 2–100 read from memory.





Apache Spark: Architecture & RDDs

Spark History and the In-Memory Insight

Origin: UC Berkeley AMPLab (2009–2013)

- **2009**: Matei Zaharia, PhD student at UC Berkeley, observes MapReduce is inefficient for iterative ML and interactive queries
- **2010**: First Spark paper (HotCloud 2010) — *"Cluster Computing with Working Sets"*
- **2012**: RDD paper (NSDI 2012) formalises fault tolerance via **lineage** — Best Paper Award
- **2013**: Spark donated to Apache Software Foundation; Databricks founded
- **2014**: Spark sets world record in 100 TB sort benchmark (23 min vs. 72 min for Hadoop)
- **2016–present**: Default big data engine at Netflix, LinkedIn, Alibaba, Uber

The Core Insight

> *"We introduce the concept of **resilient distributed datasets** (RDDs), a distributed memory abstraction that lets programmers perform in-memory computations on large clusters in a fault-tolerant manner."*
>
> — Zaharia et al., NSDI 2012

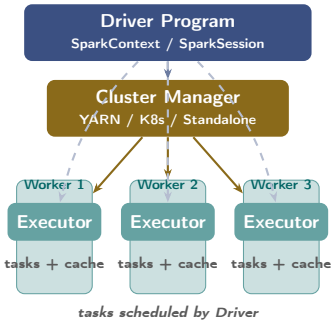
Why It Matters for Bangladesh

Bangladesh's largest ML use cases (fraud detection at bKash, disease prediction at DGHS, ride-demand forecasting at Pathao) all involve iterative algorithms over large datasets. These are **exactly** the workloads where Spark outperforms

Spark Architecture

Three-Tier Architecture

1. **Driver Program**: runs the user's `main()` function; creates `SparkContext`; builds the DAG of transformations; coordinates execution
2. **Cluster Manager**: allocates resources across the cluster (YARN, Kubernetes, Mesos, or Spark Standalone)
3. **Executors**: JVM processes launched on Worker Nodes; execute tasks; store cached RDD partitions in memory



SparkContext vs. SparkSession

In Spark 1.x, `SparkContext` was the entry point. In Spark 2.x+, `SparkSession` unifies SQL, DataFrames, and RDDs under one entry point. `SparkContext` is still

Resilient Distributed Datasets (RDDs)

Definition (Zaharia et al., 2012)

An RDD is a **read-only, partitioned collection of records** that can only be created by:

1. Transforming an existing RDD (e.g., `map`, `filter`)
2. Loading data from stable storage (e.g., HDFS, S3)

Five key properties:

- **Immutable**: never modified in place; transformations create new RDDs
- **Partitioned**: split across Executors for parallel processing
- **Lazy**: transformations only execute when an action is called

What "Resilient" Actually Means

Traditional fault tolerance = **replicate data** (3 copies, like HDFS).
RDD fault tolerance = **re-derive data** from its lineage (the computation graph).

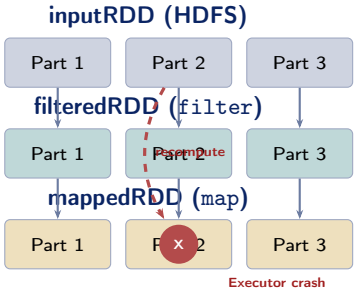
If Partition 3 of an RDD is lost:

1. Spark reads the original HDFS input for that partition
2. Re-applies the same sequence of transformations
3. Result: the lost partition is regenerated

No data replication overhead. The computation is the backup.

The Key Trade-off

RDD Lineage: The Fault-Tolerance Mechanism



Recovery Path

1. Spark detects lost Part 2
2. Traces lineage: mappedRDD was derived from filtered-RDD
   was derived from inputRDD
3. Re-reads inputRDD Part 2 from HDFS
4. Re-applies `filter` then `map`
5. Part 2 restored **without** restarting the whole job

Transformations, Actions, and Lazy Evaluation

Transformations vs. Actions

Transformations (Lazy)

Build the **computation DAG** but do not execute immediately. Return a new RDD.

Transformation	What it does
`map(f)`	Apply `f` to each element
`filter(f)`	Keep elements where `f` is True
`flatMap(f)`	Map then flatten result
`distinct()`	Remove duplicates
`reduceByKey(f)`	Aggregate values per key
`join(other)`	Inner join on key
`groupByKey()`	Group values by key
`cache()`	Persist RDD in memory

Actions (Eager)

Trigger execution of the DAG. Return a value to the Driver or write to storage.

Action	What it returns
`collect()`	All elements (list)
`count()`	Number of elements
`first()`	First element
`take(n)`	First `n` elements
`reduce(f)`	Single aggregate value
`saveAsTextFile`	Write to HDFS/disk

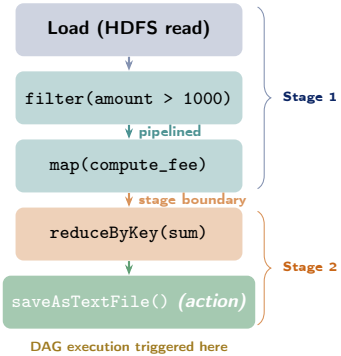
Rule:

Lazy Evaluation: Why It Matters

What Lazy Evaluation Means

When you call `rdd.filter(f).map(g)`, Spark does **nothing** yet. It only records the computation graph (DAG). When you call `.count()` or `.collect()`, Spark:

1. Analyses the full DAG
2. Applies the **Catalyst optimizer** (for DataFrames) or Spark's DAG scheduler
3. **Pipelines** compatible transformations: runs `filter` and `map` in a single pass over each partition — never writing intermediate results to disk
4. Executes only the partitions needed



Concrete Benefit

Narrow vs. Wide Dependencies: Stage Boundaries

Narrow Dependencies (no shuffle)

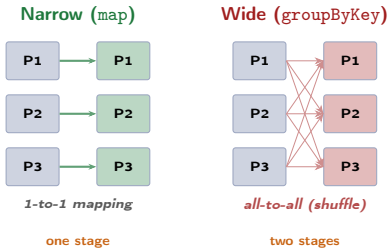
Each output partition depends on **at most one** input partition. Can be **pipelined** within a single stage.

- `map`, `filter`, `flatMap`
- `union`, `mapPartitions`
- `coalesce` (reduce partitions, no shuffle)

Recovery: if a partition is lost, recompute only that partition from its single parent partition.

Wide Dependencies (requires shuffle)

Each output partition may depend on **multiple** input partitions. Data must move across the network (shuffle). Creates a **stage boundary**.



Performance Tip

Prefer `reduceByKey` over `groupByKey`: `reduceByKey` applies a local combine before shuffling (like a Combiner in MapReduce), greatly reducing network traffic.

PySpark Programming

PySpark: Word Count and Beyond

Classic Word Count in PySpark

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("WordCount") \
    .getOrCreate()
sc = spark.sparkContext

# Load from HDFS
lines = sc.textFile(
    "hdfs:///data/corpus/*.txt")

# Transformations (lazy)
words   = lines.flatMap(
    lambda l: l.split())
pairs   = words.map(
    lambda w: (w, 1))
counts  = pairs.reduceByKey(
    lambda a, b: a + b)

# Action: write result
counts.saveAsTextFile(
    "hdfs:///output/wordcount/")
```

bKash Fraud Detection Pipeline

```
# Load transaction records (Parquet)
txns = sc.textFile(
    "hdfs:///data/bkash/txns/")

# Parse CSV fields
def parse(line):
    f = line.split(",")
    return (f[0],       # txn_id
            f[1],       # sender
            float(f[2]))# amount

parsed = txns.map(parse)

# Flag suspicious: > 50000 BDT
# to new recipient in first txn
suspicious = parsed.filter(
    lambda r: r[2] > 50000.0)

# Count by sender
alerts = suspicious \
    .map(lambda r: (r[1], 1)) \
    .reduceByKey(lambda a,b: a+b) \
```

Spark SQL and DataFrames

Spark SQL and the DataFrame API

What is a DataFrame?

A **DataFrame** is a distributed collection of data organised into **named columns** with a schema — like a table in a relational database or a Pandas DataFrame, but distributed across a Spark cluster.

- Built on top of RDDs internally
- Uses the **Catalyst optimizer**: rewrites query plans for maximum efficiency (predicate pushdown, join reordering, constant folding)
- Supports both **SQL** syntax and **Python/Scala** method chaining
- Reads Parquet, ORC, CSV, JSON, Hive tables, JDBC

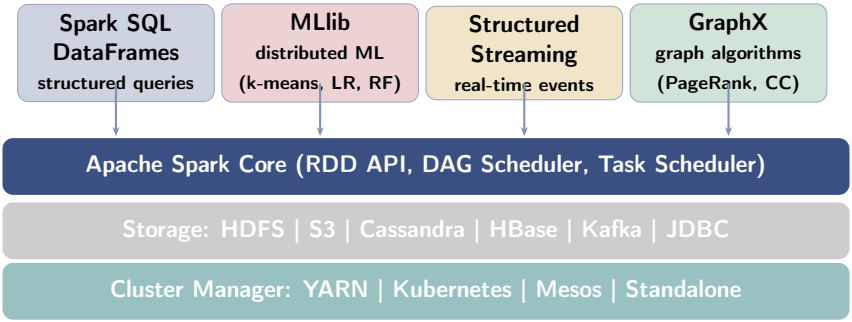
DataFrame vs RDD

DGHS Health Analytics (Spark SQL)

```
# Load Parquet health records
df = spark.read.parquet(
    "hdfs:///data/dghs/records/")

# Register as SQL view
df.createOrReplaceTempView(
    "health_records")

# Query: district-wise disease count
result = spark.sql("""
  SELECT district,
         disease_code,
         COUNT(*) AS cases
  FROM   health_records
  WHERE  year = 2024
    AND  status = 'confirmed'
  GROUP BY district, disease_code
  ORDER BY cases DESC
  LIMIT 20
""")

# Write result as CSV
```

The Spark Ecosystem

Spark Ecosystem: One Engine, Many Workloads



MLlib: Distributed Machine Learning

Supports classification, regression, clustering, dimensionality reduction, and collaborative filtering. All algorithms are designed to run in-memory across the cluster, leveraging Spark's RDD caching for iterative training.

Structured Streaming

Treats a stream as a continuously-appended table; uses the same DataFrame/SQL API as batch. Supports event-time windowing, watermarks for late data, and exactly-once processing with Kafka.

MapReduce vs. Spark: Head-to-Head

MapReduce vs. Spark: Complete Comparison

Dimension	MapReduce	Spark
Intermediate storage	Disk (HDFS) after every step	Memory (spill to disk if needed)
Programming model	Map + Reduce only	map, filter, join, groupBy, SQL, ML, stream
Languages	Java (primary)	Python, Scala, Java, R
Iterative algorithms	Slow (disk per iteration)	Fast (cache RDD; 10–100×)
Fault tolerance	Re-execute task from HDFS	Recompute from lineage
Stream processing	Not supported natively	Structured Streaming (near real-time)
Interactive queries	Not feasible (>30 s)	Spark Shell / Zeppelin (sub-second)

Ivy League Alignment

Spark in Top University Curricula

Stanford CS246 (Leskovec) — Mining of Massive Datasets

The entire modern portion of CS246 is taught in **PySpark**:

- **RDD lineage** is covered as a formal directed acyclic graph (DAG) with typed edges (narrow vs. wide)
- **PageRank on Spark**: students implement the iterative algorithm and observe the 100× speedup vs. MapReduce because the graph adjacency matrix is cached
- **Homework assignments**: all submitted as PySpark notebooks run on a shared Spark cluster; students must justify every `cache()` call with a cost estimate

Stanford exam question: *"Given the following Spark DAG, identify all stage boundaries and estimate the total shuffle volume for a 1 TB input dataset with 1,000 partitions."*

MIT 6.830 — Database Systems

MIT frames Spark as a **query engine with deferred execution**:

- Catalyst optimizer = a relational algebra rewriter
- DataFrame.explain() shows the physical plan — same concept as `EXPLAIN` in SQL databases
- Spark SQL closes the gap between MapReduce-style systems and MPP databases

CMU 15-721 — Advanced DB Systems (Pavlo)

CMU benchmarks Spark SQL against

Student Assessment Pool

Assessment—MCQ Set A

CLO3

Instructions

Select the single best answer. Correct answers **bolded** for instructor reference.

Q1. What makes Spark significantly **faster than MapReduce** for iterative algorithms?

(a) Spark uses more CPU cores per node
(b) Spark writes intermediate results to faster SSDs
(c) **Spark caches intermediate data as RDDs in memory**
(d) Spark skips the shuffle phase entirely

Q2. How does Spark achieve **fault tolerance** in RDDs *without* full data replication?

(a) Checkpointing to HDFS every 60 seconds
(b) **Tracking lineage and recomputing lost partitions from parent RDDs**
(c) Copying each RDD partition twice across two Executors
(d) Using a write-ahead log on the Driver node

Instructor Notes

Q1: Option (a) is a common distractor — Spark uses the same hardware as MapReduce. The key is

Q3. Which of the following is **NOT** a component of the Spark ecosystem?

(a) MLlib (b) GraphX (c) Structured Streaming (d) **HBase**

Q4. In Spark, a "transformation" such as `map()` or `filter()` is:

(a) Immediately executed when called and materialised to memory
(b) **Lazily evaluated — only executed when a subsequent action is called**
(c) Automatically persisted to HDFS after execution
(d) Only available in the Scala API, not in PySpark

Instructor Notes

Q3: HBase (option d) is a separate NoSQL database (covered in Week 4) that runs on HDFS. It can be used as a *data source* for Spark but is not part of the Spark ecosystem itself. **Q4:** This is the fundamental "lazy evaluation" concept. Students who answer (a) have a critical misconception that leads to inefficient Spark programs. Show `rdd.filter().count()` vs. just `rdd.filter()` to demonstrate.

Assessment—True / False

CLO3

Instructions

State True or False and provide a **one-sentence justification** for full marks.

Statement	Answer
1. Spark can only run on top of Hadoop YARN and has no other deployment option.	**False**
2. RDDs are mutable — individual elements can be modified in place after creation.	**False**
3. Spark's DataFrame API provides schema-aware, named-column operations similar to SQL tables.	**True**
4. MapReduce is generally better than Spark for machine learning workloads that require many iterations over the same dataset.	**False**

Assessment—Descriptive Questions *CLO3*

Instructions

Answer each question in **100–150 words**. Include a diagram where indicated.

D1. Explain the concept of a **Resilient Distributed Dataset (RDD)**. What does *"resilient"* mean in this context? How does Spark recover a lost RDD partition without replication?

D2. Differentiate between **Spark transformations** and **actions**. Give two concrete examples of each. Why does Spark not execute transformations immediately?

D3. Describe Spark's **lazy evaluation model**. What is the advantage of deferring computation until an action is called? Use the example `rdd.filter(x > 1000).first()` to illustrate.

Instructions

Answer in **200–300 words** with step-by-step reasoning and quantified estimates.

A1. Iterative I/O Cost Analysis:

You need to train a K-Means clustering model over **50 GB** of bKash transaction data with **100 iterations**. Assume HDFS read speed = 500 MB/s and RAM cache read speed = 5,000 MB/s.

- Calculate the total HDFS I/O for MapReduce (all 100 iterations)
- Calculate the total I/O for Spark (50 GB fits in cluster RAM)
- Estimate the wall-clock time for each approach
- Under what conditions would Spark NOT outperform MapReduce?

A2. RDD Failure Recovery Trace:

A Spark job computes `input` → `filteredRDD` → `mappedRDD` → `reducedRDD` over 10 partitions. Mid-execution, the Executor holding partitions 3 and 7 of `reducedRDD` crashes. Trace the complete recovery sequence: what does the Driver detect, which partitions of which RDDs need to be recomputed, and what is the worst-case recovery cost if `reducedRDD` involves a **wide** dependency (`groupByKey`)?

Textbook & Reading References

Primary Textbooks for Week 6

- **[TW]** Tom White. *Hadoop: The Definitive Guide*, O'Reilly, 4th ed., 2015. **Ch. 2, 19**
  - Ch. 2: MapReduce (baseline for comparison) · Ch. 19: Spark (appendix)
- **[SA]** Acharya & Chellappan. *Big Data and Analytics*, Wiley, 2015. **Ch. 7**
  - Ch. 7: Introduction to Apache Spark
- **[LRU]** Leskovec, Rajaraman & Ullman. *Mining of Massive Datasets*, 3rd ed. **Ch. 2.4**
  - Ch. 2.4: Spark's model; extensions beyond MapReduce

Supplementary (Covered in Week 6, Part 2 — Seminal Papers)

- Zaharia, M. et al. (2010). **Spark: Cluster Computing with Working Sets.** *USENIX HotCloud 2010.*
- Zaharia, M. et al. (2012). **Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing.** *USENIX NSDI 2012.* **Best Paper Award. [PRIMARY SEMINAL PAPER]**
- Karau, H. et al. *Learning Spark*, O'Reilly, 2015 (1st ed. freely available as PDF).

Questions & Discussion

Week 6, Part 1 | CSE 4345 Big Data Analytics

"We introduced RDDs to provide a programming model that allows users to explicitly control how data is cached in memory across operations — a capability that simply did not exist in MapReduce."

— Matei Zaharia, NSDI 2012

Next Lecture: Week 6, Part 2

Seminal Paper 1: Zaharia et al. (2010) — *"Spark: Cluster Computing with Working Sets"*

Seminal Paper 2: Zaharia et al. (2012) — *"Resilient Distributed Datasets"*

Case Study: Databricks logistic regression benchmark (2014) — 100× vs. Hadoop MapReduce on the same cluster

Reading before Part 2: [SA] Ch. 7 · Zaharia et al. (2012) RDD paper (see references)